Eseményvezérelt alkalmazások: 3. gyakorlat

A munkafüzet bevezet minket a modell-nézet alapú grafikus alkalmazás fejlesztésbe. A modell komponens felel az üzleti logikáért, független a nézettől és a grafikus könyvtáraktól (*Windows Forms*). Állapotváltozásait a nézet felé eseményeken keresztül jelzi. A nézet komponens a megjelenítésért és a megjelenítéshez kapcsolódó logikáért felel elsődlegesen.

Dokumentum statisztikák (grafikus felülettel)

Készítsünk grafikus felületet az 1. gyakorlaton elkészített, szöveges állományokban a szavak előfordulását számláló alkalmazáshoz, melyen megjelenítjük a betöltött fájl szövegét, a szavak előfordulásainak számát, valamint még néhány hasznos információt!

Az új projekt létrehozása során válasszuk a **Windows Forms App** sablont! A projekt neve legyen `DocuStatView`, a *solution* neve pedig `DocuStat`!

Ne a *Windows Forms App (.NET Framework)* sablont válasszuk, ugyanis az .NET 10 helyett a korábbi .NET Frameworkre épülő projektet fog létrehozni!

Nevezzük át a `Form1.cs` fájlt `DocuStatDialog.cs`-re!

A Designerben helyezzük fel a következő vezérlőket az ablakra:

- `MenuStrip`: az ablak tetején, a címsor alatt helyezkedjen el, ez lesz az alkalmazás menüsávja. A vezérlőre jobb klikkelve válasszuk ki az **Edit Items...** menüpontot. Az **Add** gombbal adjunk új menüelemet (`MenuItem`) a menüsávhoz.
  - A jobb oldalon látható részben be tudjuk állítani a menüelem nevét (`(Name)`), ez legyen `fileMenu`, és szövegét (`Text`), ez legyen `File`. A `fileMenu`-t kijelölve keressük meg a tulajdonságok között a `DropDownItems` elemet, és a `(Collection)`-re kattintva válasszuk ki a mellette megjelenő három pontot. Ezzel egy hasonló menüt kapunk, ahol a `fileMenu`-höz adhatunk új menüelemeket az **Add** gombbal.
    - Adjunk a `fileMenu`-höz egy új elemet, ennek neve (`(Name)`) legyen `openFileDialogMenuItem`, felirata (`Text`) pedig legyen `Open file dialog`.
    - Egy második menüelem neve legyen `resetFilterMenuItem`, a felirata pedig legyen `Reset filter`.

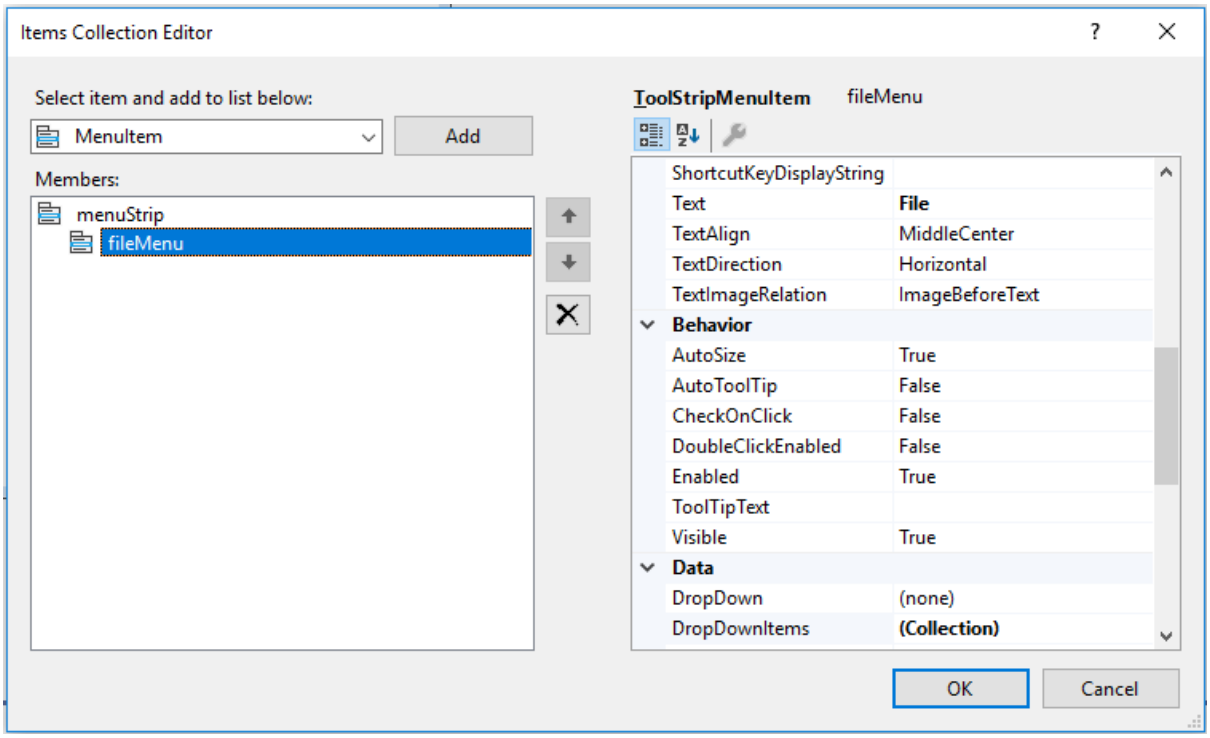


Figure 1: Menüpont hozzáadása a menüsávhoz.

- `TextBox`: ebben fogjuk megjeleníteni a szöveges fájl tartalmát. Helyezzük az ablak bal oldalára, és méretezzük szélességét úgy, hogy kb. az ablak felét foglalja el. A *Solution Explorer* alatt megjelenik a Designerben aktuálisan kijelölt elemhez tartozó tulajdonságok listája (`Properties`), amelyben módosítani tudjuk a vezérlő viselkedését, eseménykezelőket adhatunk az eseményeihez stb. A `TextBox (Name)` tulajdonságát írjuk át `textBox`-ra, majd a `ScrollBars` tulajdonságot állítsuk `Vertical`-ra, hogy görgethető legyen a mező. Állítsuk igazra a `MultiLine` tulajdonságot, hogy a magassága több soros is lehessen, ezután növeljük meg a magasságát. A `ReadOnly` tulajdonságot is állítsuk igazra, hogy a felületen keresztül ne legyen szerkeszthető a szöveg.
- `ListBox`: ebben jelenítjük meg a szavakat és előfordulásaik számát. Helyezzük az ablak jobb oldalára, és méretezzük át úgy, hogy mérete a bal oldali `TextBox` méretével megegyezzen. A `Properties` ablakban adjuk neki a `listBoxCounter` nevet.
- `Label`: Adjunk a felülethez az ábrán látható módon 3 címkét ezekkel a feliratokkal: `Minimum word length:`, `Minimum word occurrence:`, `Ignored words:`. Majd adjunk még további 6 címkét az alábbi név-felirat tulajdonságokkal:
  - `labelCharacters` - `Character count:`,
  - `labelNonWhitespaceCharacters` - `Non-whitespace characters:`,
  - `labelSentences` - `Sentence count:`,
  - `labelProperNouns` - `Proper noun count:`,
  - `labelColemanLiauIndex` - `Coleman-Liau Index:`,
  - `labelFleschReadingEase` - `Flesch Reading Ease:`.
- `NumericUpDown`: ezekben fogjuk beállítani a minimális szóhosszot és minimális előfordulások számát. Mindegyik `Minimum` tulajdonságát állítsuk 1-re. Név tulajdonságuk legyen `spinBoxMinLength` és `spinBoxMinOccurrence`. Az előfordulások `Maximum` tulajdonsága legyen 500!
- `TextBox`: A szavak számlálásából kizárandó szavakat ide lehet majd beírni vesszővel elválasztva, szóköz nélkül. Neve legyen `textBoxIgnoredWords`.

A teljes ablak kijelölésével láthatjuk a `Properties` ablakban az ablak tulajdonságait. Legyen a fejléc szövege (`Text`) `Document statistics`! A `FormBorderStyle` tulajdonságot állítsuk `FixedSingle`-re, a `MaximizeBox` tulajdonságot pedig hamisra, így az ablak fix méretű lesz.

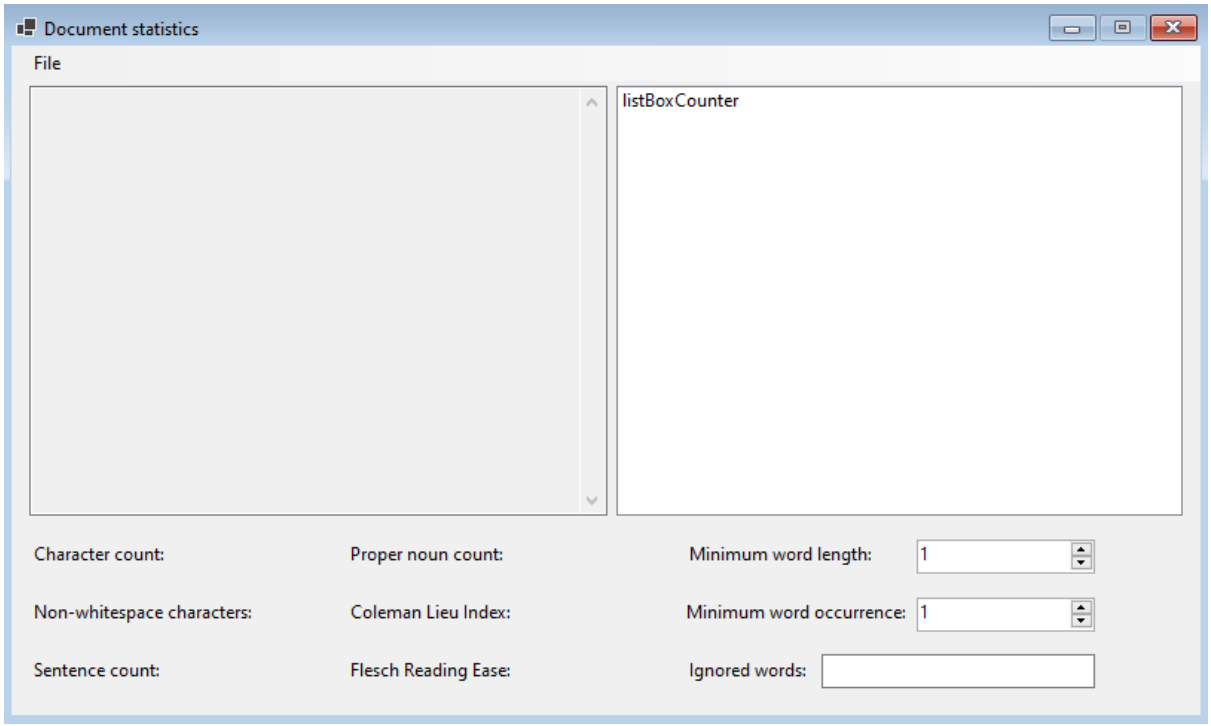


Figure 2: Az alkalmazáshoz tartozó ablak nézete a Designerben.

Modell aggregálása a nézetben ^{KM}

Mivel fel fogjuk használni az 1. gyakorlat `DocumentStatistics` osztályát, ezért először hozzá kell férnünk az azt tartalmazó projekthez is. Jobb klikkeljünk a Solution nevére, **Add –> Existing Project...** . Válasszuk ki az 1. gyakorlat `DocuStat.csproj` nevű projekt fájlját. Ezt követően a *DocuStatView* projekt nevére jobb klikkeljünk, **Add –> Project Reference...** . Pipáljuk be a *DocuStat* projektet, majd okézzuk le az ablakot.

Az eseményvezérelt paradigmának megfelelően a modell eseményekkel jelzi az állapotváltozásait. A `DocumentStatistics` osztályhoz három egyedi eseményt adunk hozzá:

```
public event EventHandler? FileContentReady;
public event EventHandler<TextStatisticsReadyEventArgs>? TextStatisticsReady;
public event EventHandler? FilteredWordCountChanged;
```

- `FileContentReady`: a szöveges fájl tartalmának beolvasása megtörtént.
- `TextStatisticsReady`: a metrikák kiszámítása megtörtént.
- `FilteredWordCountChanged`: megváltozott a `FilteredWordCount`, azaz a megjelenítendő szólista. Ezt kiválthatja a szűrő átállítása (`MinWordLength`, `MinWordOccurrence`, `IgnoredWords`), de a fájl betöltése is, hiszen olyankor maga a szóstatisztika épül újra.

Adatot szállító események: egyedi `EventArgs`

Az első és a harmadik esemény csak *jelzést* küld: a nézet az értesítés hatására kiolvassa a modell megfelelő tulajdonságát (*pull* stílus). A `TextStatisticsReady` viszont **magával viszi** az adatot (*push* stílus). Ehhez származtassunk az `EventArgs` osztályból egy sajátot a `Model` könyvtárban, `TextStatisticsReadyEventArgs` néven, amely csak olvasható tulajdonságokban tárolja a metrikákat, és a konstruktorán keresztül kapja meg őket.

```
public class TextStatisticsReadyEventArgs : EventArgs
{
    public int CharacterCount { get; }
    public int NonWhiteSpaceCharacterCount { get; }
    // ...
```

```
    public TextStatisticsReadyEventArgs(int characterCount, /* ... */)
    {
        CharacterCount = characterCount;
        // ...
    }
}
```

Melyiket mikor érdemes használni? A *pull* stílus egyszerűbb, és akkor kényelmes, ha a nézetnek sok, egymással összefüggő adatra van szüksége (mint a szűrt szólistánál). A *push* stílus viszont pontosan megmondja, **mi** változott, és az eseménykezelő az `EventArgs`-ból mindig érvényes adatot kap: nem kell a modellre hivatkoznia, így a `_documentStatistics` esetleges `null` volta sem okoz gondot.

Az események kiváltása

Szükségünk lesz privát metódusokra, amelyekkel az eseményeket kiváltjuk:

```
private void OnFileContentReady()
{
    FileContentReady?.Invoke(this, EventArgs.Empty);
}

private void OnTextStatisticsReady()
{
    TextStatisticsReady?.Invoke(this, new TextStatisticsReadyEventArgs(
        CharacterCount, NonWhiteSpaceCharacterCount, SentenceCount,
        ProperNounCount, ColemanLiauIndex, FleschReadingEase));
}

private void OnFilteredWordCountChanged()
{
    FilteredWordCountChanged?.Invoke(this, EventArgs.Empty);
}
```

Az első kettőt hívjuk meg a `Load()` metóduson belül a megfelelő helyeken:

- `OnFileContentReady()`: a szöveges fájl tartalmának beolvasását követően.
- `OnTextStatisticsReady()`: a metrikák kiszámítását követően.

Esemény csak tényleges változáskor

Az `OnFilteredWordCountChanged()`-et egyrészt a `Load()` metódus végén hívjuk meg (a szóstatisztika ekkor épült újra), másrészt a három szűrő tulajdonság *setter*-éből. Alakítsuk át az 1. gyakorlaton felvett automatikus tulajdonságokat úgy, hogy legyen saját adatmezőjük, és **csak akkor** váltsanak ki eseményt, ha az érték ténylegesen megváltozott:

```
public int MinWordLength
{
    get => _minWordLength;
    set
    {
        if (_minWordLength == value)
            return; // a szűrő nem változott, nincs mit jelezni

        _minWordLength = value;
        OnFilteredWordCountChanged();
    }
}
```

Az `IgnoredWords` esetében a normalizált listákat a `SequenceEqual()` metódussal hasonlíthatjuk össze. (A normalizálás során az üres szavakat is dobjuk el a `Where(w => w.Length > 0)` szűréssel, különben egy üres szövegdoboz vagy egy záró vessző is "változásnak" számítana.)

> Ez a látszólag apró részlet fontos: a nézet a szűrő mindhárom tulajdonságát beállítja majd minden egyes vezérlő-változásnál. Az őrfeltétel nélkül egyetlen billentyűleütés három eseményt, és így a lista háromszori újraépítését okozná. Az ilyen programozói hibából fakadó fölösleges körök az eseményvezérelt alkalmazásokban tipikus teljesítményproblémát okozhatnak.

Nyissuk meg a `DocuStatDialog.cs` fájlt (nem a `DocuStatDialog.Designer.cs`-t)! Ebben a fájlban írjuk meg a menüelemekhez tartozó eseménykezelőket, amelyek a fájlbeolvasást és a statisztika elkészítését fogják végezni, és kitöltik a `textBox`-ot és a `listBoxCounter`-t. A usingok közé vegyük fel az `ELTE.DocuStat.Model` névteret! Vegyünk fel egy privát `DocumentStatistics?` típusú adattagot (`_documentStatistics`).

Fájl betöltése *EM*

Definiáljuk az `openFileDialogMenuItem`-hez tartozó eseménykezelőt! Az eseménykezelő egy privát, `void` visszatérési típusú metódus, amely a küldő objektumot és egy eseményargumentumot vár paraméterül. A neve legyen `OpenDialog`.

```
private void OpenDialog(object? sender, EventArgs e)
```

A fájlt most egy fájltallózó dialóguson keresztül fogjuk megnyitni. Használjuk ehhez az `OpenFileDialog` osztályt. A dialógus kezdeti könyvtárát az `InitialDirectory`, a megengedett fájltípusokat a `Filter` tulajdonságon keresztül állíthatjuk be. Ha azt szeretnénk, hogy minden megnyitáskor a kezdeti könyvtár jelenjen meg, állítsuk a dialógus `RestoreDirectory` tulajdonságát igazra.

A dialógus `ShowDialog()` metódusát meghívva megjelenítjük a fájltallózót. Ha itt sikeresen kiválasztottunk egy fájlt, akkor példányosítsunk egy új `DocumentStatistics` objektumot, melynek konstruktor paramétere legyen az `openFileDialog` `FileName` propertyje. Iratkozzunk fel a modell mindhárom eseményére az `UpdateFileContent`, `UpdateTextStatistics` és `UpdateWordList` eseménykezelőkkel. Ezt követően az `UpdateWordFilter` hívásával adjuk át a friss modellnek a felületen esetleg már beállított szűrőt, majd hívjuk meg a `_documentStatistics` `Load()` metódusát. A szólistát nem kell külön feltöltenünk: a `Load()` a végén maga váltja ki a `FilteredWordCountChanged` eseményt. Ne feledjük, hogy itt el kell kapnunk az esetleges kivételt. A `catch` ágban jelenítsünk meg egy `MessageBox`-ot a kivétel szövegével (`Message`), majd térjünk vissza.

```
private void OpenDialog(object? sender, EventArgs e)
{
    using (OpenFileDialog openFileDialog = new OpenFileDialog())
    {
        openFileDialog.InitialDirectory = "C:\\";
        openFileDialog.Filter = "txt files (*.txt)|*.txt|All files (*.*)|*.*";
        openFileDialog.RestoreDirectory = true;

        if (openFileDialog.ShowDialog() == DialogResult.OK)
        {
            try
            {
                _documentStatistics = new DocumentStatistics(openFileDialog.FileName);
                _documentStatistics.FileContentReady += UpdateFileContent;
                _documentStatistics.TextStatisticsReady += UpdateTextStatistics;
                _documentStatistics.FilteredWordCountChanged += UpdateWordList;

                UpdateWordFilter(this, EventArgs.Empty);

                _documentStatistics.Load();
            }
            catch (System.IO.IOException ex)
            {
                MessageBox.Show("File reading is unsuccessful!\n" + ex.Message,
                "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }
        }
    }
}
```

A `UpdateFileContent` eseménykezelőben megspórolhatjuk a szöveg ismételt betöltését abban az esetben, ha ugyanazt a fájlt ugyanazzal a tartalommal próbáljuk betölteni, mint ami már a `textBox`-ban van. Vizsgáljuk meg, hogy a `_documentStatistics.FileContent` egyezik-e a `textBox` tartalmával (`Text`), feltéve, hogy előbbi nem üres (amit a `String.IsNullOrEmpty()` metódussal tudunk ellenőrizni)! Ha a betöltendő szöveg egyezik a korábbival, térjünk vissza.

A `textBox` `Text` tulajdonságának adjuk értékül a `FileContent` tartalmát, így megjelenik a `textBox`-ban a fájlból beolvasott szöveg. A `listBoxCounter` `Items` kollekciójának tartalmát töröljük a `Clear()` metódussal.

```
private void UpdateFileContent(object? sender, EventArgs e)
{
    if (_documentStatistics?.FileContent == textBox.Text)
        return;

    textBox.Text = _documentStatistics?.FileContent;
    listBoxCounter.Items.Clear();
}
```

A `textBox` alatt elhelyezkedő címkéket is frissíteni kell, hogy már a kiszámított metrikák értékei is megjelenjenek rajtuk. Ezt a `UpdateTextStatistics` eseménykezelővel fogjuk elérni, amelynek eseményargumentuma most már nem az általános `EventArgs`, hanem a saját `TextStatisticsReadyEventArgs` típusunk. Az `e` argumentum megfelelő tulajdonságait felhasználva állítsuk át a címkék `Text` tulajdonságát úgy, hogy az eredeti szövegek mellett az eredmények is látszódjanak.

```
private void UpdateTextStatistics(object? sender, TextStatisticsReadyEventArgs e)
{
    labelCharacters.Text = $"{_prefixCharacters} {e.CharacterCount}";
    // ...
```

```
}
```

> Figyeljük meg, hogy itt már nincs szükség a `_documentStatistics!` null-forgiving operátorra: az eseményargumentum mindig érvényes adatot hordoz.

Az `OpenDialog`-ot kössük össze a konstruktorban az `openFileDialogMenuItem` `Click` eseményével.

```
openFileDialogMenuItem.Click += OpenDialog;
```

Eseménykezelőt a `-=` operátorral vehetünk le egy eseményről.

Szó statisztikák megjelenítése és szűrése ^{EM}

Az 1. gyakorlaton a szűrés paraméterei (`MinWordLength`, `MinWordOccurrence`, `IgnoredWords`) és a szűrt eredmény (`FilteredWordCount`) már a modellbe kerültek. A nézetnek így nincs más dolga, mint

1. a felület vezérlőinek változásait **beírni** a modellbe, és
2. a modell `FilteredWordCountChanged` eseményére feliratkozva **kiolvasni** a `FilteredWordCount`-ot – függetlenül attól, hogy a változást a szűrő átállítása vagy egy új fájl betöltése okozta.

Ez az a pont, ahol az eseményvezérelt architektúra kézzelfoghatóvá válik: nem kell külön menüpont a statisztika "kiszámítására", a lista magától, azonnal követi a beállításokat.

A felület változásainak átvezetése a modellbe

Definiáljunk egy `UpdateWordFilter` eseménykezelőt, amely a három vezérlő aktuális értékét a modellbe írja! A `Convert.ToInt32()` metódussal alakíthatjuk egész számmá a `NumericUpDown` értékét; az ignorált szavakat viszont már nem kell előkészítenünk, mert a modell `IgnoredWords` setter-e maga normalizálja őket.

```
private void UpdateWordFilter(object? sender, EventArgs e)
{
    if (_documentStatistics == null)
        return;

    _documentStatistics.MinWordLength = Convert.ToInt32(spinBoxMinLength.Value);
    _documentStatistics.MinWordOccurrence = Convert.ToInt32(spinBoxMinOccurrence.Value);
    _documentStatistics.IgnoredWords = textBoxIgnoredWords.Text.Split(',');
}
```

Kössük rá ezt az eseménykezelőt mindhárom vezérlő megfelelő eseményére a konstruktorban:

```
spinBoxMinLength.ValueChanged += UpdateWordFilter;
spinBoxMinOccurrence.ValueChanged += UpdateWordFilter;
textBoxIgnoredWords.TextChanged += UpdateWordFilter;
```

> Figyeljük meg, hogy az `UpdateWordFilter` **mindhárom** tulajdonságot beállítja, akkor is, ha csak az egyik vezérlő változott. Ezért volt fontos, hogy a modell *setter*-ei csak tényleges változás esetén váltsanak ki eseményt: enélkül egyetlen felhasználói művelet három `FilteredWordCountChanged` eseményt, és így a lista háromszori felépítését okozná.

A lista frissítése a modell eseményére

Definiáljunk egy `UpdateWordList` eseménykezelőt, amely a `listBoxCounter` tartalmát a modell `FilteredWordCount` tulajdonságából tölti fel! A `listBoxCounter` vezérlő `BeginUpdate()` metódusát meghívva kezdhetjük a párok kiírását. A párokon végig iterálva adjuk hozzá az `Items`-hez szövegként a párok kulcsát és értékét, majd az `EndUpdate()` metódussal zárjuk le a `listBoxCounter` szerkesztését.

```
private void UpdateWordList(object? sender, EventArgs e)
{
```

```
    if (_documentStatistics == null)
        return;

    listBoxCounter.Items.Clear();
    listBoxCounter.BeginUpdate();
    foreach (var pair in _documentStatistics.FilteredWordCount)
    {
        listBoxCounter.Items.Add(pair.Key + ": " + pair.Value);
    }
    listBoxCounter.EndUpdate();
}
```

A lista frissítését a `BeginUpdate()` és az `EndUpdate()` keretébe zárása nélkül is elvégezhetjük, ez esetben azonban minden elem hozzáadása a megjelenítés frissítését váltja ki, amely így egy villogó hatást okozhat a felhasználói felületen. Élő, gépelés közben frissülő listánál ez különösen zavaró lenne.

A szűrő alaphelyzetbe állítása

A `resetFilterMenuItem` menüponthoz készítsünk egy `ResetFilter` eseménykezelőt! Ha még nem olvastunk be fájlt, dobjunk fel egy `MessageBox`-ot, amely tájékoztat erről, majd térjünk vissza.

Egyébként egyszerűen állítsuk vissza a **vezérlőket** az alapértelmezett értékükre. A vezérlők eseményei ekkor kiváltják az `UpdateWordFilter`-t, az a modellbe ír, a modell pedig a `FilteredWordCountChanged` eseménnyel frissíti a listát. Így a lista frissítésének egyetlen útvonala van, függetlenül attól, hogy a felhasználó a spinboxot használta, gépelt, vagy a menüpontra kattintott.

```
private void ResetFilter(object? sender, EventArgs e)
{
    if (_documentStatistics == null)
    {
        MessageBox.Show("No file is loaded!", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    spinBoxMinLength.Value = spinBoxMinLength.Minimum;
    spinBoxMinOccurrence.Value = spinBoxMinOccurrence.Minimum;
    textBoxIgnoredWords.Text = string.Empty;
}
```

A `ResetFilter` eseménykezelőt az előzőhöz hasonlóan kössük rá a `resetFilterMenuItem` vezérlő `Click` eseményére.

Az alkalmazás osztálydiagramja:

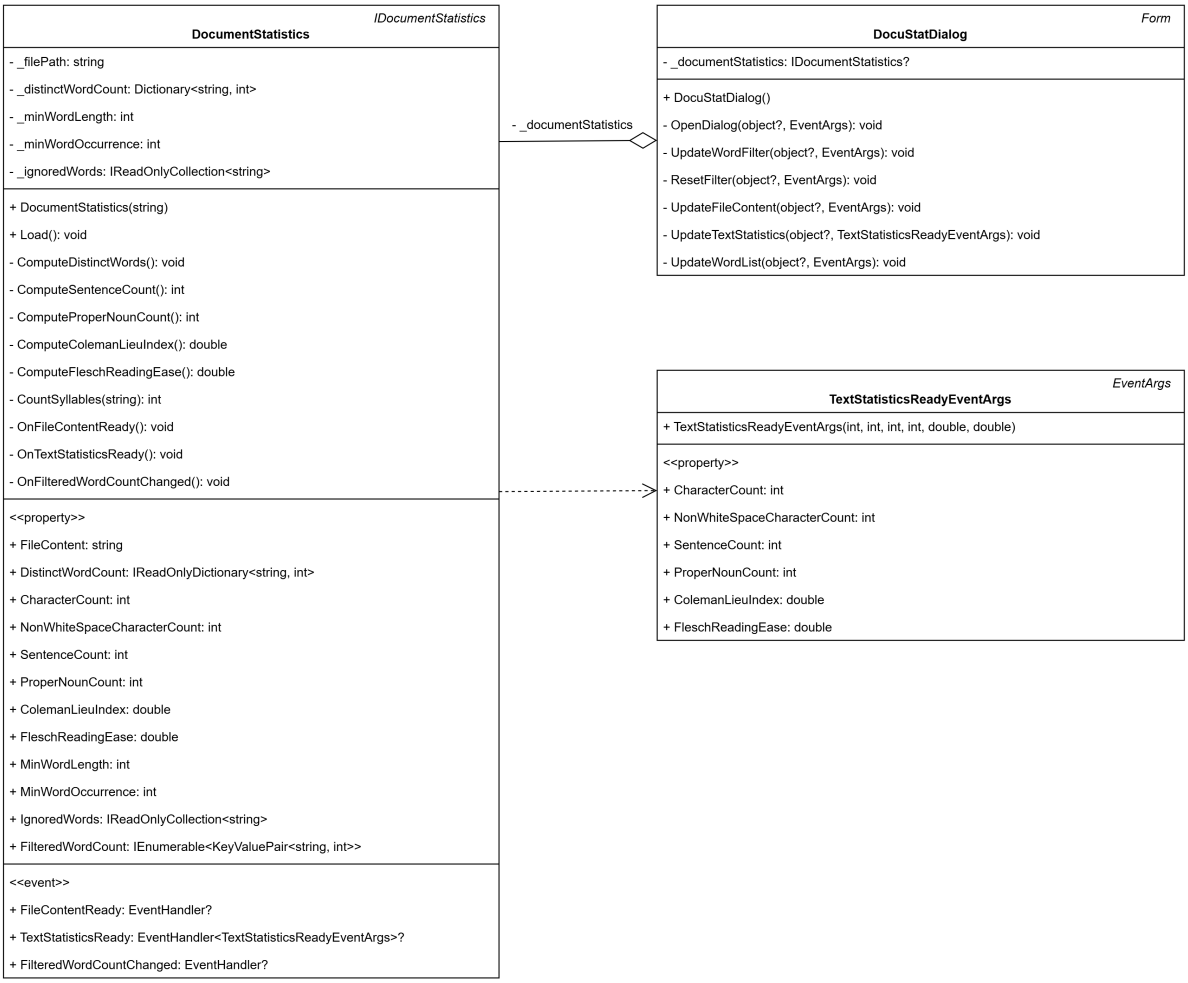


Figure 3: Az alkalmazáshoz tartozó osztálydiagram.